Exercise 2

Joshua John E. Santiago

2026-09-30

```
a <- 17
b <- 5
a + b
```

```
## [1] 22
```

```
a - b
```

```
## [1] 12
```

```
a * b
```

```
## [1] 85
```

```
a / b
```

```
## [1] 3.4
```

```
# TODO: Using the same a and b, compute and print a%%b and a%/%b
# a %% b = remainder of a/b
# a %/% b = whole-number (integer) division
```

```
a %% b
```

```
## [1] 2
```

```
a %/% b
```

```
## [1] 3
```

```
# TODO: Print 2 raised to the power of 10 using the ^ operator
```

```
2^10
```

```
## [1] 1024
```

Question #1: Run `17 / 5`, `17 %% 5`, and `17 %/% 5` side by side. In your own words, explain how these three results relate to each other mathematically.

Answer: Running `17 / 5`, `17 %% 5`, and `17 %/% 5` yields `3.4`, `2`, and `3`. These numbers represent exact division, the remainder, and the integer quotient. They relate mathematically because multiplying the quotient by the divisor and adding the remainder reconstructs the original number (`(3 * 5) + 2 = 17`). Additionally, dividing the remainder by the divisor (`2 / 5 = 0.4`) produces the exact decimal portion of the first result.

Question #2: Predict the output of `10 %% 3` and `10 %/% 3` before running the code. Were you correct? If not, explain what you misunderstood.

Answer: Yes, my prediction was correct. `10 %% 3` produces `1` (the remainder) and `10 %/% 3` produces `3` (the whole-number quotient).

```r
# TODO: Use typeof() to check the type of each of x, y, z, w, v
# TODO: Use class() to check the class of each of x, y, z, w, v
# TODO: Use mode() to check the mode of each of x, y, z, w, v

x <- 42
y <- 42L
z <- "42"
w <- TRUE
v <- 3 + 2i

typeof(x)
```

```
## [1] "double"
```

```r
typeof(y)
```

```
## [1] "integer"
```

```r
typeof(z)
```

```
## [1] "character"
```

```r
typeof(w)
```

```
## [1] "logical"
```

```r
typeof(v)
```

```
## [1] "complex"
```

```r
class(x)
```

```
## [1] "numeric"
```

```r
class(y)
```

```
## [1] "integer"
```

```r
class(z)
```

```
## [1] "character"
```

```r
class(w)
```

```
## [1] "logical"
```

```r
class(v)
```

```
## [1] "complex"
```

```r
mode(x)
```

```
## [1] "numeric"
```

```r
mode(y)
```

```
## [1] "numeric"
```

```r
mode(z)
```

```
## [1] "character"
```

```r
mode(w)
```

```
## [1] "logical"
```

```
mode(v)
```

```
## [1] "complex"
```

Question #3: `x <- 42` and `y <- 42L` look almost identical, but `typeof(x)` and `typeof(y)` give different answers. Explain the difference in your own words.

Answer: The difference is how R stores the numbers in computer memory. Even though they look the same, R defaults to storing `x <- 42` as a `double`, which is a decimal/floating-point format capable of holding fractional parts. By contrast, the `L` suffix on `y <- 42L` forces R to store the number explicitly as an `integer`, which is restricted to whole numbers and consumes less memory.

Question #4: Is `typeof()` ever different from `class()` and from `mode()` for the same object? Look at your results above and explain any case where this happens (or, if you did not see one, explain what you would expect for a matrix or factor).

Answer: Yes, `typeof()`, `class()`, and `mode()` can yield completely different results for the same object. Even in the simple vectors above, `x <- 42` has `typeof(x)` as `"double"`, while `class(x)` and `mode(x)` are both `"numeric"`. They also diverge when handling structured objects like factors (where `typeof()` is `"integer"` for the underlying level codes, while `class()` is `"factor"`).

```
# TODO: Convert num_string to a numeric value using as.numeric()
# Then check its type with typeof(), class(), and mode()

num_string <- "123"
num <- as.numeric(num_string)
typeof(num)
```

```
## [1] "double"
```

```
class(num)
```

```
## [1] "numeric"
```

```
mode(num)
```

```
## [1] "numeric"
```

```
# TODO: Convert the number 0 and the number 1 to logical using as.logical()
# What do you get?

as.logical(0)
```

```
## [1] FALSE
```

```
as.logical(1)
```

```
## [1] TRUE
```

Answer: I got `FALSE` and `TRUE`.

```
# TODO: Try converting bad_string to numeric using as.numeric()
# Run this chunk and observe the warning message and the result

bad_string <- "hello"
as.numeric(bad_string)
```

```
## Warning: NAs introduced by coercion
```

```
## [1] NA
```

Question #5: When you ran the conversion-fail chunk, R produced `NA` and a warning instead of stopping with an error. Why do you think R is designed to give a warning here rather than crashing the whole script? What are the risks of this behavior if you don't read your warnings?

Answer: R gives a warning instead of an error because real data is messy, and one bad value shouldn't crash the whole script. Returning `NA` lets the conversion finish for all valid values while marking the ones that failed, and the warning makes sure the problem isn't silent. However, if you ignore the warning, `NA` values can spread silently through your analysis. Calculations like `mean()` or `sum()` return `NA` (or, with `na.rm = TRUE`, quietly drop those values), so your results can be wrong or based on less data than you think.

Question #6: `as.logical(0)` and `as.logical(1)` convert to `FALSE` and `TRUE`. What do you think `as.logical(2)` and `as.logical(-1)` will produce? Test it, then explain the rule R seems to be following.

Answer: Both will produce `TRUE`. R treats `0` as `FALSE` and any non-zero number—whether positive or negative—as `TRUE`, so `as.logical(2)` and `as.logical(-1)` both evaluate to `TRUE`.

```
as.logical(2)
```

```
## [1] TRUE
```

```
as.logical(-1)
```

```
## [1] TRUE
```

Part D

```
# TODO: Print first_name and last_name using print() and/or cat()

first_name <- "Juan"
last_name <- "Dela Cruz"
print(first_name)
```

```
## [1] "Juan"
```

```
print(last_name)
```

```
## [1] "Dela Cruz"
```

```
# TODO: Use paste() to combine first_name and last_name into a full name
# TODO: Use paste0() to do the same, and compare the two results

full_name1 <- paste(first_name, last_name)
full_name2 <- paste0(first_name, last_name)
full_name1
```

```
## [1] "Juan Dela Cruz"
```

```
full_name2
```

```
## [1] "JuanDela Cruz"
```

```
# TODO: Use paste() with the argument sep = "-" to join first_name and
# last_name with a dash instead of a space

paste(first_name, last_name, sep = "-")
```

```
## [1] "Juan-Dela Cruz"
```

```
# TODO: Use paste0() to build the sentence:
# "Juan Dela Cruz is 21 years old." by combining full_name, age, and text
```

```r
full_name <- paste(first_name, last_name)
age <- 21
paste0(full_name, " is ", age, " years old.")
```

```
## [1] "Juan Dela Cruz is 21 years old."
```

Question #7: `paste()` and `paste0()` produced different results for the same two strings. Explain exactly why, referencing what each function does by default.

Answer: `paste()` and `paste0()` gave different results because they use different default separators. `paste()` has `sep = " "`, so it inserts a single space between the strings it joins, giving `"Juan Dela Cruz"`. `paste0()` is equivalent to `paste()` with `sep = ""`, so it inserts nothing between them, giving `"JuanDela Cruz"`.

Question #8: In the strings-numbers chunk, you combined a character string with a numeric variable (`age`) using `paste0()` without first converting `age` to a string. Did this cause an error? Explain what `paste0()` does internally to make this work, and how it relates to the type conversions you practiced in Part C.

Answer: No, it did not cause an error. `paste0()` implicitly converts each of its arguments to character using `as.character()` before joining them. So the numeric `21` quietly becomes the string `"21"`, and the pieces are then concatenated into one string.

Part E

```r
# TODO: Predict, then test: what is the result of "5" + 5 in R?
# Run it and read the error message carefully.

"5" + 5
```

```
## Error in `"5" + 5`:
## ! non-numeric argument to binary operator
```

```r
# TODO: Now fix the code above so that "5" + 5 correctly evaluates to 10
# using an appropriate conversion function.

as.numeric("5") + 5
```

```
## [1] 10
```

Question #9: Some programming languages automatically convert `"5" + 5` into `10` without an error. R does not. Based on what you've learned about R's data types in this lab, why do you think R requires an explicit conversion instead of guessing what the programmer meant?

Answer: R doesn't guess because `"5"` is text (`character`), not a number, and `+` only works on numeric or logical values. R also can't know whether you meant `10` (addition) or `"55"` (joining text). Making you write `as.numeric("5")` forces you to state your intent, so mistakes show up as a clear error instead of a silently wrong answer.

Question #10: Suppose a classmate writes a script that reads a column of numbers from a text file. The column looks numeric, but when they try `sum()` on it, R throws an error. Based on everything in this lab (types, `typeof()`/`class()`/`mode()`, conversion, warnings), write a short paragraph explaining what likely went wrong and the steps you'd take to diagnose and fix it.

Answer: The column was likely read into R as a `character` vector rather than a numeric one, which often happens if even a single cell in the text file contains a stray letter, symbol, or whitespace. Because `sum()` cannot add text strings, R throws an invalid `'type' (character)` error. To diagnose this, I would first check the column using `typeof()` and `class()` to confirm it is stored as `"character"`. Next, I would convert it using `as.numeric()` and carefully watch for an `"NAs introduced by coercion"` warning, which would

reveal if any messy non-numeric entries failed to convert. Once converted (and after inspecting any `NA` values), `sum(..., na.rm = TRUE)` will work properly.

Conclusion

In 3–4 sentences, summarize what you learned about how R handles different data types, and why understanding `typeof()`/`class()` and conversion functions matters before doing arithmetic or combining data.

Answer: In this lab, I learned that R stores every value as a specific data type (such as double, integer, character, logical, or complex), and that `typeof()`, `class()`, and `mode()` reveal how a value is stored and treated. Functions like `as.numeric()` and `as.logical()` convert between types, though invalid conversions return `NA` with a warning, whereas `paste0()` converts numbers to text automatically and `+` refuses to mix text and numbers. Checking types and converting explicitly before doing arithmetic or combining data matters because values that look numeric may actually be text, leading to errors or silently incorrect results. Reading warnings and verifying data types keeps issues visible at the source instead of letting them corrupt an entire analysis.